

I had my first encounter with research in 2019 over the course of a rainy summer in Dublin. During long walks on the Irish coast, I reflected on how I learned to program on BlueJ, an educational Java IDE, in my high school's introductory programming course that made heavy use of the tool. Two years later, I was analyzing millions of logs from BlueJ's Blackbox data collection project to assess an update to compiler error messages. Throughout the summer, we found statistically significant differences, such as an increase in average displayed error messages per hour from 6.6 to 9.7 ($p < 0.001$) [1]. The implications nagged at me: millions of novice programmers learned how to code using BlueJ, and this change enacted by the BlueJ designers had significantly altered their debugging experience. This version, BlueJ 4, was released a few months before I first downloaded the IDE in my high school computer science class. On my walks that summer, I wondered: what other aspects of my journey as a developer would be altered if some design decision had a different outcome?

My curiosity around this question would shape both my undergraduate studies and my professional career as a software engineer. Back at the University of Michigan, I was determined to continue CS education research. Through Google's exploreCSR program, I joined Professor Westley Weimer's lab to assist Ph.D. candidate Madeline Endres (now professor at the University of Massachusetts-Amherst). I initially helped run skill-based interventions for a study comparing the effects of spatial awareness training versus technical reading. The results of this study indicated that CS-focused technical reading training could be beneficial for novices' programming skills [2]. From this study, I became curious about other skills, not explicitly taught in standard CS curricula, that could positively impact programming outcomes. Driven by the wide range in debugging efficacy I observed in office hours as a TA for my school's Data Structures and Algorithms course, I sought to better understand how one becomes effective at debugging code, and potentially identify replicable behaviors that could be taught to students. The following summer, I analyzed a dataset of user-supplied programs from Professor Philip Guo's Python Tutor. As I examined consecutive program submissions, I noticed drastic changes where entire functions materialized with different variable names, formatting, and logic, suggesting users had copied code from external sources. This raised a critical question: how were users finding these sources? Through several brainstorming sessions with my advisors, this question evolved into a study investigating help-seeking behavior among novice and expert programmers, focusing on Google and Stack Overflow as the primary sites of interaction.

The process of executing this study from conception to publication taught me the rigor and perseverance necessary to conduct meaningful empirical research. With funding support and IRB approval, I recruited 40 participants and facilitated hours of Zoom study sessions. The project ultimately spanned over a year from ideation to paper submission, with weeks spent performing qualitative analysis on debugging session recordings to identify behavioral patterns. My efforts resulted in a first-author paper published in ICSE-SEET 2022, in which we found a significant correlation between programming expertise and Stack Overflow search efficacy during code debugging, and observed that experts and novices exhibited different search behaviors [3]. I began the research process overwhelmed by the sheer number of possible directions; by the end, I had learned how to identify a novel, tractable research question, structure an investigation to answer it, and execute it from end to end. I hope this study's findings can be used to inform the design of debugging training and search features to support more programmers, but the landscape of information seeking during development has evolved significantly since then with the explosion of LLMs and generative AI tools.

The rise of AI developer tools has transformed software development in many contexts, yet the question driving my research interests remains unchanged. AI tools have reduced reliance on resources like Stack Overflow [4], and programming looks drastically different than it did several years ago. Given the recent pace at which programming has evolved, I believe there is still much to understand about how AI assistance affects developer behavior. To this end, the underlying question I seek to answer remains the same: how can we design tools and experiences for developers that make software development a more positive, fulfilling experience? My curiosity surrounding this is what made me excited to see how compilation messages shaped a novice's learning, motivated me to explore how a developer can best employ Stack Overflow for debugging, and attracted me to developer experience when I became a professional software engineer. As simple as this question seems, it is open-ended and multifaceted – what defines a positive developer experience, especially now that many tasks are being offloaded to AI programming assistants, which pose their own usability challenges to developers [5]? How does this experience vary between developers, who have different backgrounds and experience levels?

In the last three years as a professional software engineer, I have witnessed a gap between industry practice and research in developer tool design. I spent several years building developer interfaces for a popular payment processing API and, more recently, libraries to support frontend development at a trading firm. Through this work, I have seen firsthand how design decisions for widely-used developer tools are made. In meetings, my teams' debates around the design of developer interfaces centered largely on what "felt more intuitive" based on our preferences as experienced engineers. This subjective approach troubled me, as I thought back to BlueJ and the significant effects its compilation change had on user behavior. I could not help but think about how a poor design choice in a widely-adopted tool could frustrate thousands of developers daily, and to what degree the design processes of widely-used developer tools were based on empirical evidence. I want to bridge this gap by conducting research that can inform the design of such tools, with the ultimate goal of reducing the amount of frustration experienced by developers. More specifically, I am interested in exploring how design choices surrounding AI-assistive technology affect developer learning curves and productivity. For example, my Stack Overflow study revealed experts and novices search differently for information; I now wonder how interaction with AI coding assistants differs with programming experience. My industry experience exposed me to how subjective the design process can be – in my PhD research, a direction that interests me is developing empirical frameworks to inform developer tool design through human studies.

With the support of the NSF CSGrad4US fellowship, I am eager to pursue this direction at *School A*, where I hope to have the opportunity to work with several faculty. I am interested in *Professor A's* work to improve software engineering experiences through various techniques, in particular the application of LLMs in software engineering tasks and understanding current development practices, as they directly relate to the questions that emerged from my industry experience. I am also drawn to *Professor B's* work on inclusivity in open source environments, as ensuring that developer tools serve programmers from varying backgrounds and experience levels is central to my research vision. Furthermore, *Professor C* and *Professor D's* research on programming language usability aligns with my curiosity about how language design choices impact developer experience. I believe *School A's* unique intersection of human-computer interaction and software engineering provides an ideal environment for my research interests, and my prior experiences with creative research, perseverance, and rigorous analysis will position me well to succeed at *School A*.

References

- [1] I. Karvelas et al., "The Effects of Compilation Mechanisms and Error Message Presentation on Novice Programmer Behavior," in Proceedings of the 51st ACM Technical Symposium on Computing Science Education (SIGCSE 2020).
- [2] M. Endres et al., "To Read or to Rotate? Comparing the Effects of Technical Reading Training and Spatial Skills Training on Novice Programming Ability," in Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE 2021).
- [3] A. Li et al., "Debugging with Stack Overflow: Web Search Behavior in Novice and Expert Programmers," in Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Software Engineering Education and Training (ICSE-SEET 2022).
- [4] B. Xu et al., "Are We Ready to Embrace Generative AI for Software Q&A?" in Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE 2023).
- [5] J. T. Liang et al., "A Large-Scale Survey on the Usability of AI Programming Assistants: Successes and Challenges," in Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE 2024).